Infrastructure, Cloud and Virtualization

Dominique.dhoutaut@univ-fcomte.fr

Introduction

- Cloud (public or private) is now ubiquitous
- It relies on a large set of **hardware** and **software** tools
- **Virtualization** is a key concept
  - It’s not a single technology
  - It has existed in some form or another for more than 40 years
  - Even in consumer grade computers, it has been present for decades

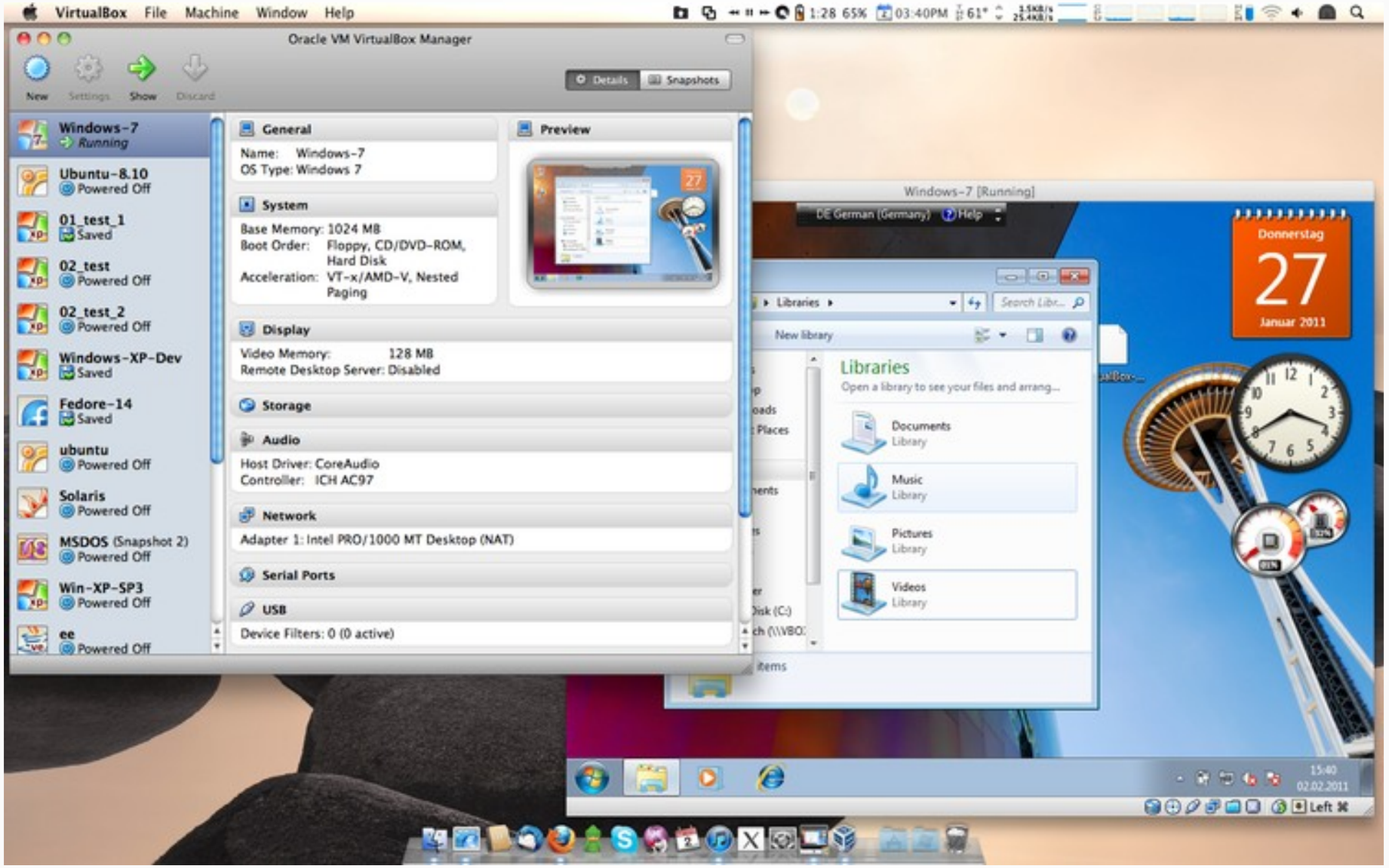
Introduction

- Objectives of this course
  - Overview and theoretical aspects of virtualization
  - State of the art / taxonomy of virtualization techniques and their use cases. All have their advantages and drawbacks, none is perfect.
  - Practice server virtualization using Proxmox to manage a set of physical and virtual computers.
  - Practice containerization
  - Broaden our view with « software » cloud as proposed by big players (like amazon AWS, microsoft azure, ...)

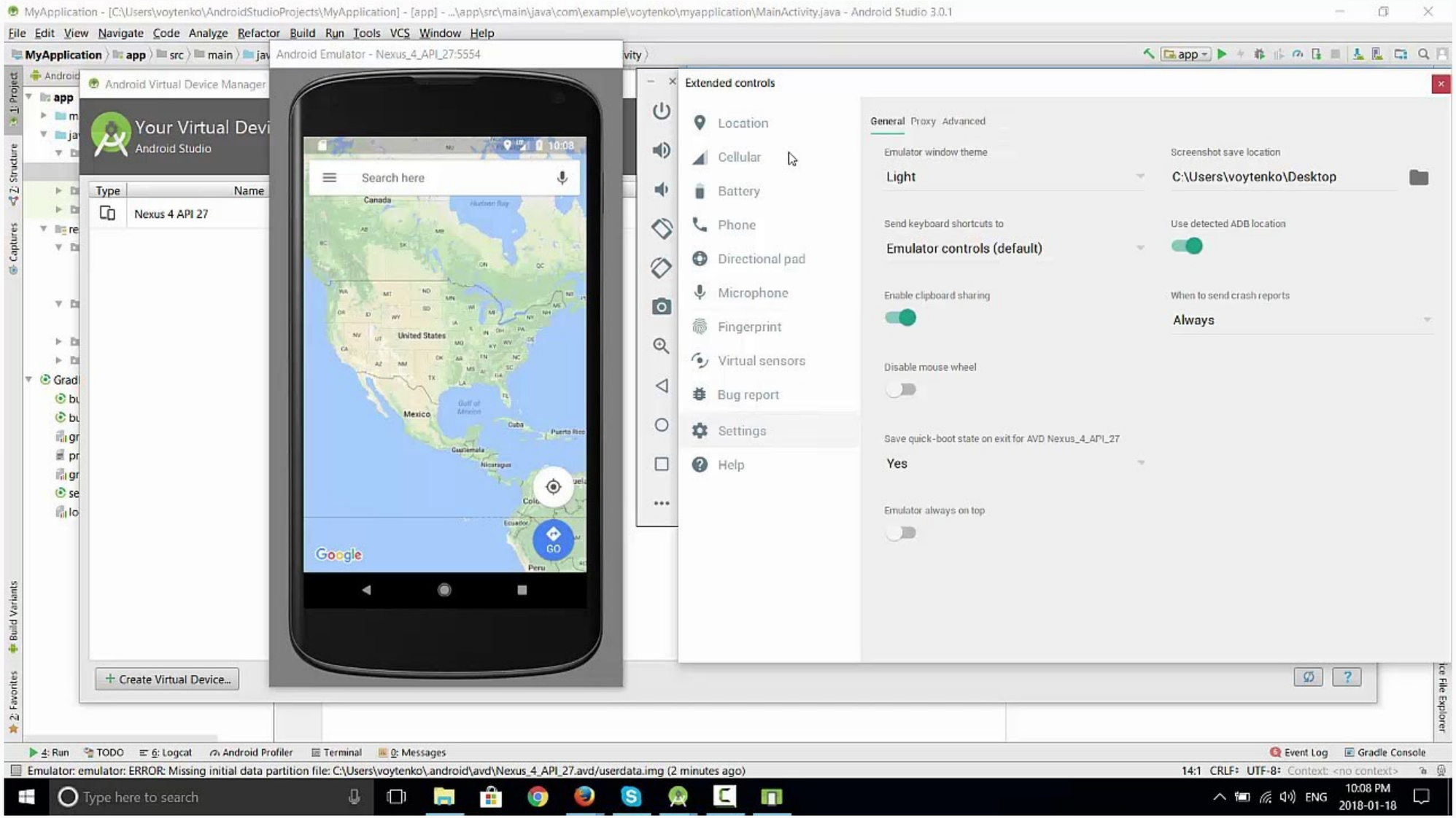
Defining Virtualization

- For now, a (voluntarily) vague definition
  - Using a **host** system to run a **guest** system.
  - The guest system may be of a different nature (operating system, hardware, …)
  - The guest system may not be aware that it is running inside a host. It may be tricked into believing it has control over the hardware.

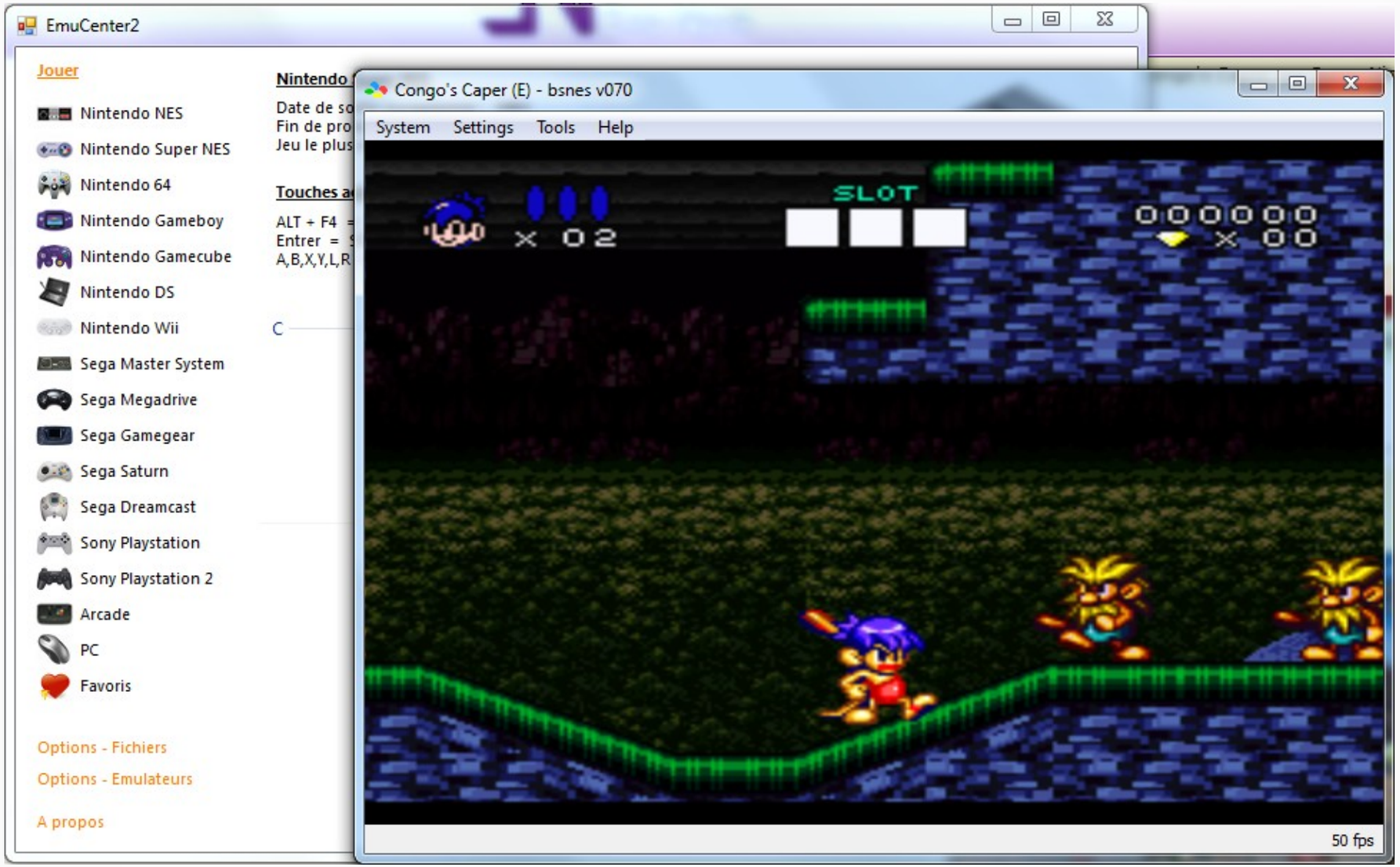
You have already used virtualization !



You have already used virtualization !



You have already used virtualization !



Benefits of virtualization

- Earn more (or spend less) money, of course !
  - To make multi-platform development easier.
  - To run old or exotic software on hardware you don’t have.
  - To optimize the usage of the hardware you possess.
  - To better isolate applications and improve security.
  - To better handle migrations and backups.
  - To allows for user mobility.
  - To play.
  - ...

Benefits of virtualization

- Earn more (or spend less) money, of course !
  - **To make multi-platform development easier.**
  - To run old or exotic software on hardware you don’t have.
  - To optimize the usage of the hardware you possess.
  - To better isolate applications and improve security.
  - To better handle migrations and backups.
  - To allows for user mobility.
  - To play.
  - ...

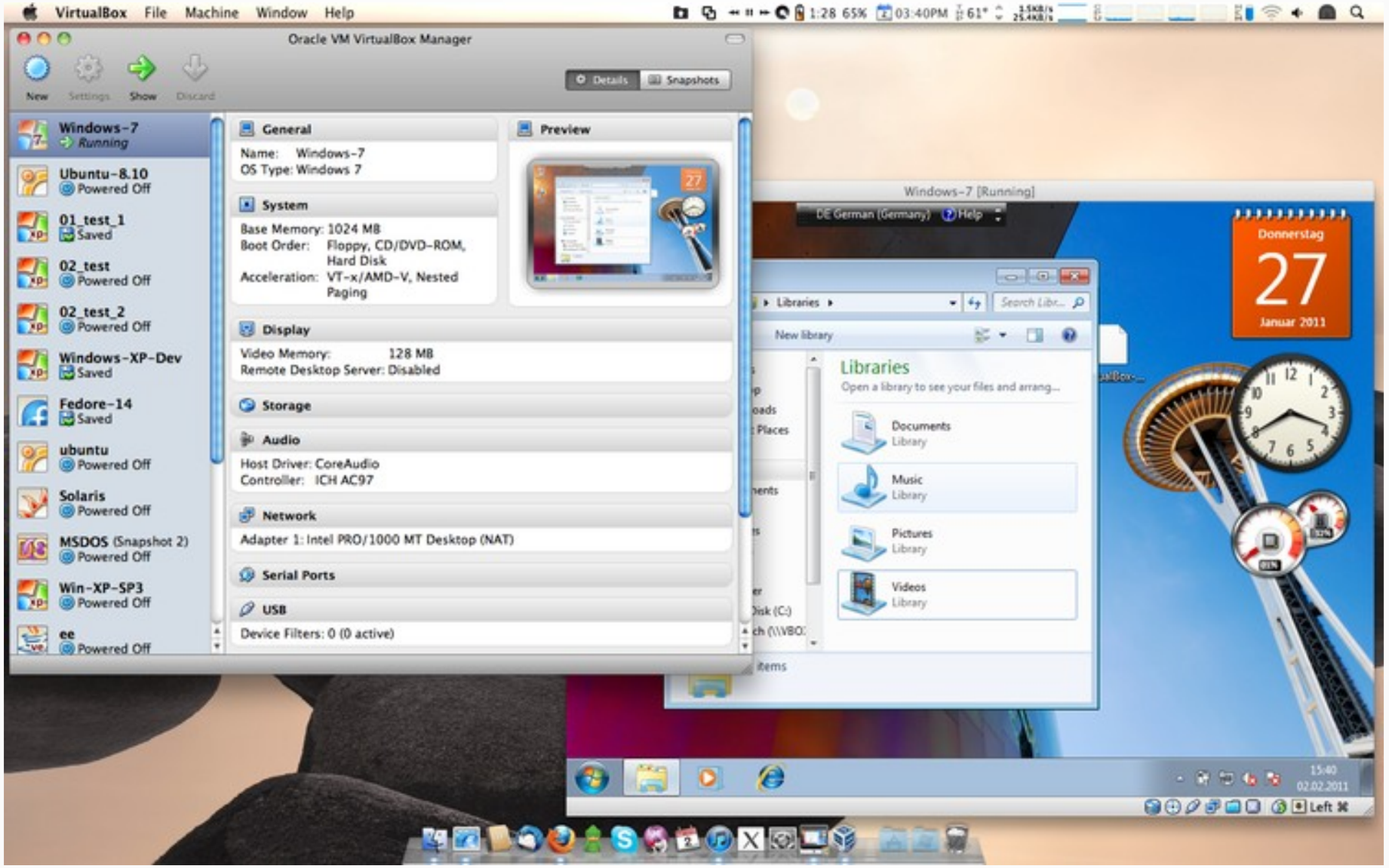
To make multi-platform development easier

- If you target many different platforms, you normally need to test on all of them. With virtualization, no need to have them all at your disposal.
- You may have different versions of the same system (think of windows with various service pack or updates, different flavours of Linux, etc.). Maintaining a test fleet for all of them is difficult.

To make multi-platform development easier

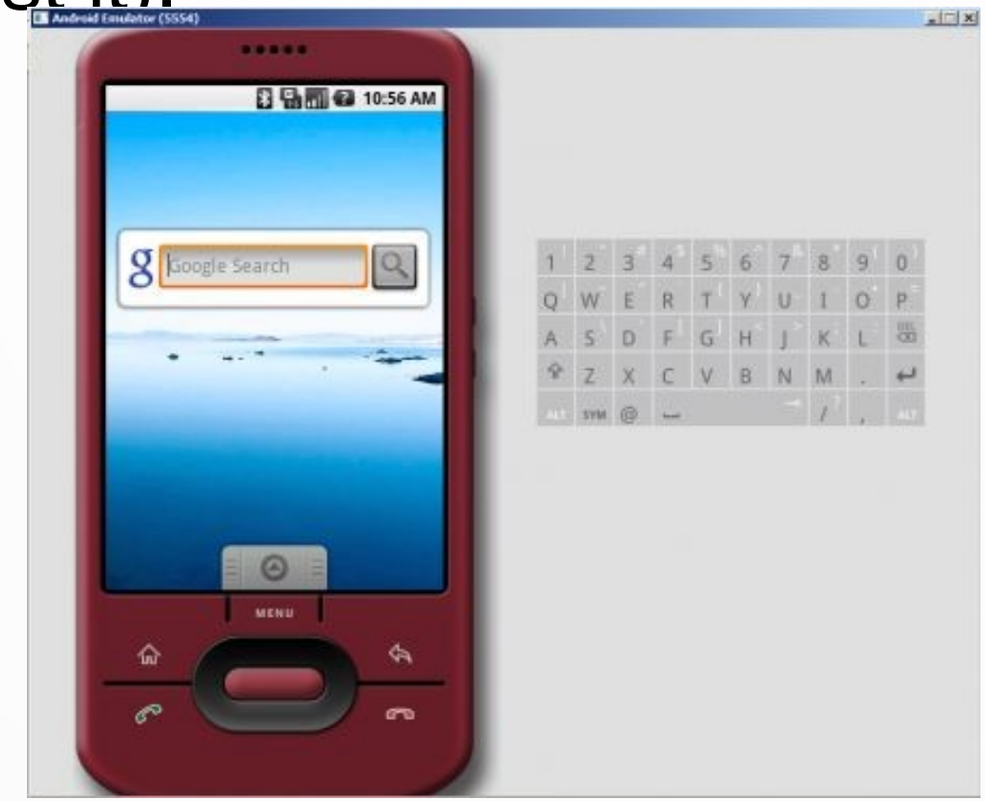
- Your tests may alter the system you run them on. You can roll back to a previous state instead of reinstalling everything !
- In this context, people often use workstation virtualization software (Virtual Box, Virtual PC, VMWare Workstation, etc.)
- You can have many virtualized systems and easily switch between them. Of course, it can be taxing on resources (RAM and CPU).

To make multi-platform development easier



To make multi-platform development easier

- Mobile development heavily rely on virtualization. It eases the workflow a lot (no need to endlessly upload your software to test it).



Benefits of virtualization

- Earn more (or spend less) money, of course !
  - To make multi-platform development easier.
  - **To run old or exotic software on hardware you don’t have.**
  - To optimize the usage of the hardware you possess.
  - To better isolate applications and improve security.
  - To better handle migrations and backups.
  - To allows for user mobility.
  - To play.
  - ...

To run old or exotic software on hardware you don’t have.

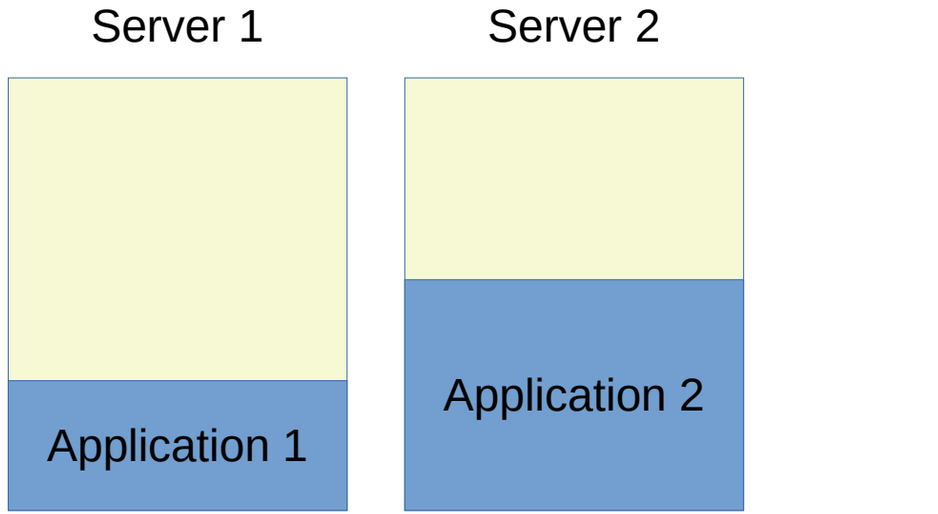
- In some application (industry, …) heavy duty equipment last much longer (50+ years) than common computers.
- As embedded computers runs old, they cannot be replaced by identical ones.
- Old software may not run on new hardware, and is often extremely costly to re-develop.
- It is better / easier to run the old software on new computers while virtualizing the old environment.

Benefits of virtualization

- Earn more (or spend less) money, of course !
  - To make multi-platform development easier.
  - To run old or exotic software on hardware you don’t have.
  - **To optimize the usage of the hardware you possess.**
  - To better isolate applications and improve security.
  - To better handle migrations and backups.
  - To allows for user mobility.
  - To play.
  - ...

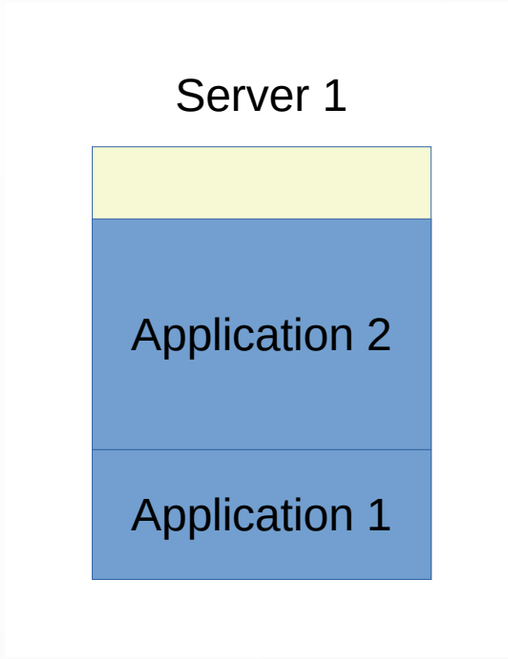
To optimize the usage of the hardware you possess

- When managing multiple applications, you usually need multiple servers, even if they are not used to the maximum of their capabilities.



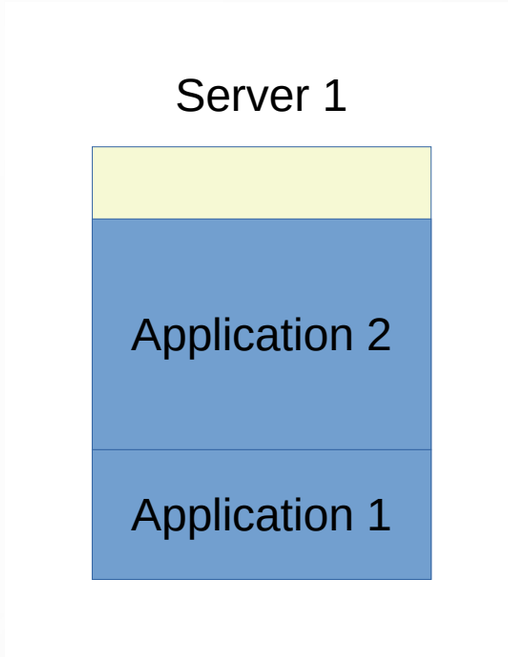
To optimize the usage of the hardware you possess

- Of course, in this case, you may want to regroup all your applications on the same server and save on the cost of the second one.
- But you want to avoid this situation for two main reasons
  - **Security**. Isolation between applications is mandatory. Security breach will happen.
  - **Resources conflict**. If the load of an application increases, it may affect the other one.



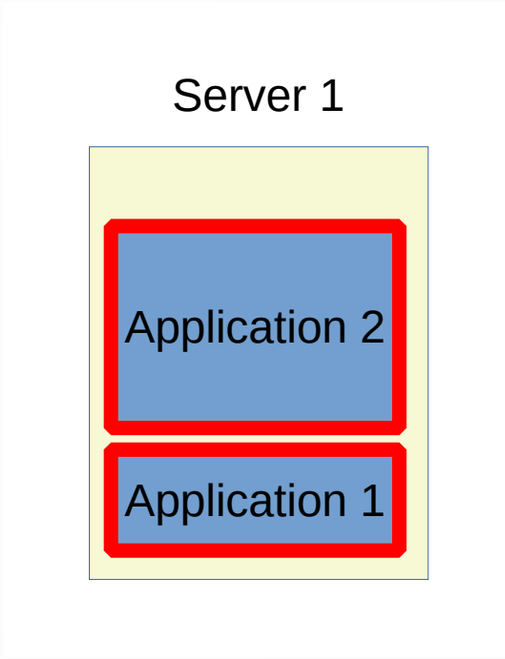
To optimize the usage of the hardware you possess

- Resources conflicts could be managed by a quota system. The operating system could be configured to limit CPU or RAM usage of a given application or user.
- However, it’s difficult to streamline this type of management.
- Also, a lot of software elements are usually shared between applications (shared libraries - DLL on windows) and sometime upgrading an application may break another one.



To optimize the usage of the hardware you possess

- Using virtual machine or container – see later for differences - per application brings multiple benefits:
  - Quotas are enforce on the virtual machine (or on the container). If you move the application, the quotas follow.
  - Each VM or container contains the software environment required by the application. Each application can have its own.

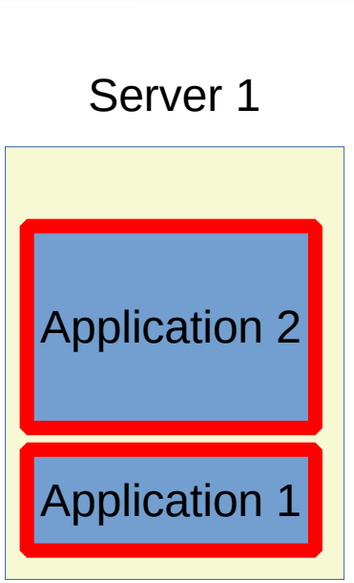


Benefits of virtualization

- Earn more (or spend less) money, of course !
  - To make multi-platform development easier.
  - To run old or exotic software on hardware you don’t have.
  - To optimize the usage of the hardware you possess.
  - **To better isolate applications and improve security.**
  - To better handle migrations and backups.
  - To allows for user mobility.
  - To play.
  - ...

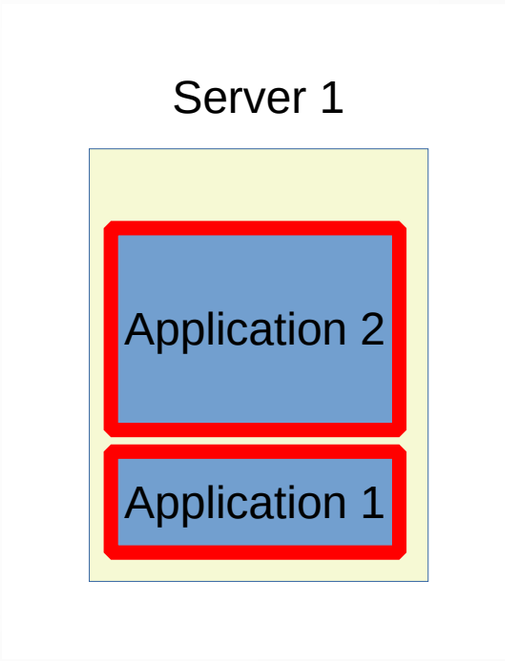
To better isolate applications and improve security

- A form of isolation is already provided by modern O.S., but viruses, backdoors, bugs and many other techniques may cause a breach.
- It is very dangerous to run multiple applications on a server, especially if some of them are accessible to the public.
- We prefer to use virtualization, **letting each application thinks it is alone on a dedicated server**.



To better isolate applications and improve security

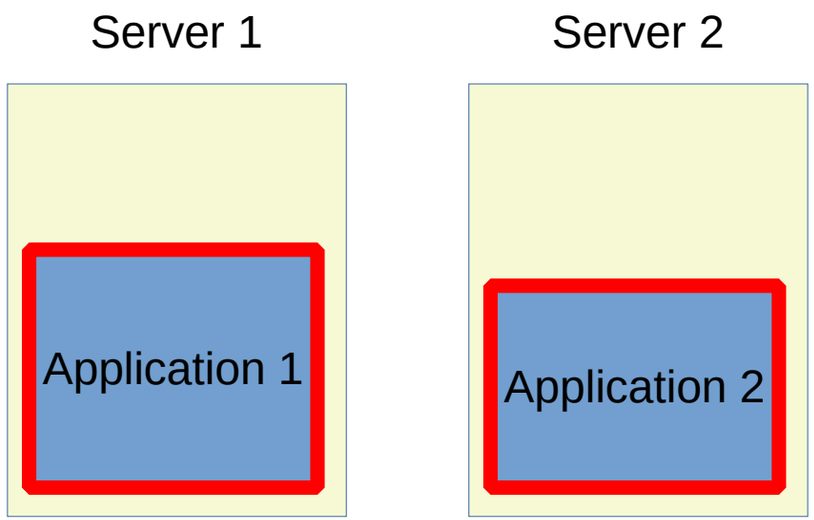
- Isolated inside their respective VMs or containers, applications have no direct way to communicate.
  - No shared memory
  - No shared file system
- They can only communicate through the network, where security can be enforce by firewalls.



To better isolate applications and improve security

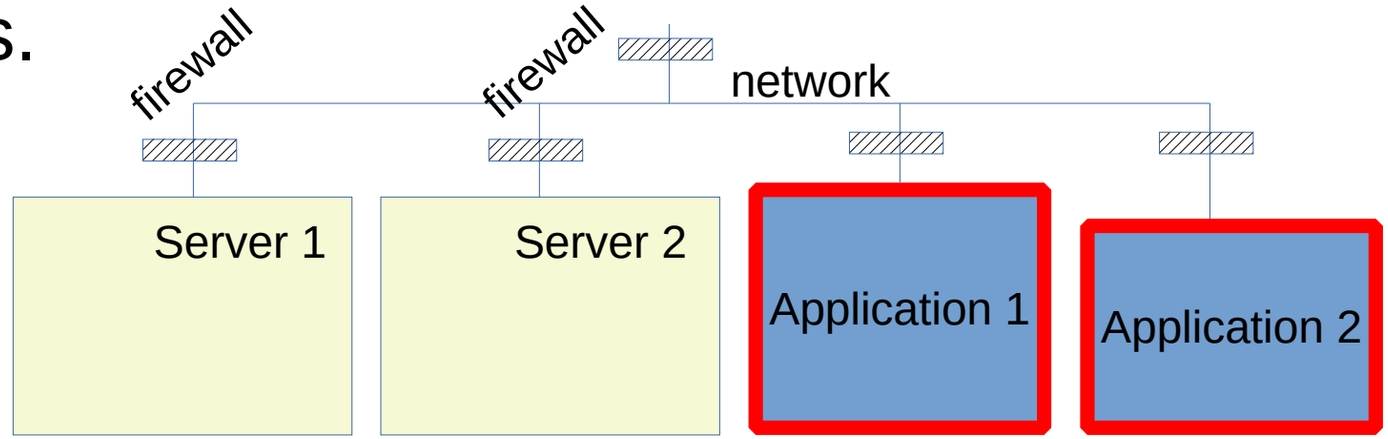
- The strength of this technique relies on the network. Each application run inside a virtual computer (or at least a container) which possesses its own network address.

- We rely on the usual network stack (routing, DHCP, swiches, ...) to reach the application through their addresses and independently of the real hardware they are running on.



To better isolate applications and improve security

- From the network point of view, physical servers and application servers are all network entities and are managed in the same way.
- Security will rely on firewall allowing or preventing communications between all those real and virtual computers.



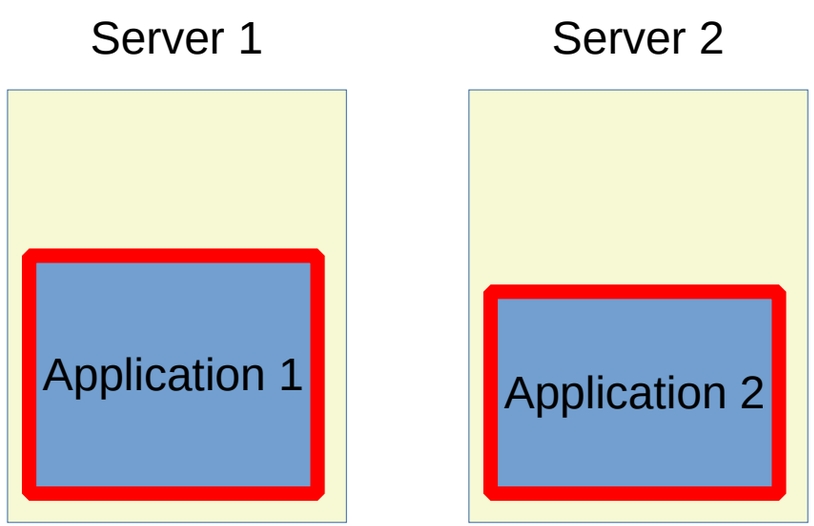
Benefits of virtualization

- Earn more (or spend less) money, of course !
  - To make multi-platform development easier.
  - To run old or exotic software on hardware you don’t have.
  - To optimize the usage of the hardware you possess.
  - To better isolate applications and improve security.
  - **To better handle migrations and backups.**
  - To allows for user mobility.
  - To play.
  - ...

To better handle migrations and backups.

- Moving applications between computers becomes easier, as they run inside a container that is standardized.

- If at some point, server 1 is overloaded, we can start server 2 and migrate an application to it
- This is also very useful for hardware maintenance



To better handle migrations and backups

- Backups of whole system are made easy.
  - Restoring a working system is consequently much faster
- Migration can be done live or offline.
- Migration can be on-site or off-site.
- Uptime goes up. It’s hard to obtain the famous 99,999% availability. A carefully tailored architecture and the use of virtualization are mandatory to reach it – more on that subject later.

Benefits of virtualization

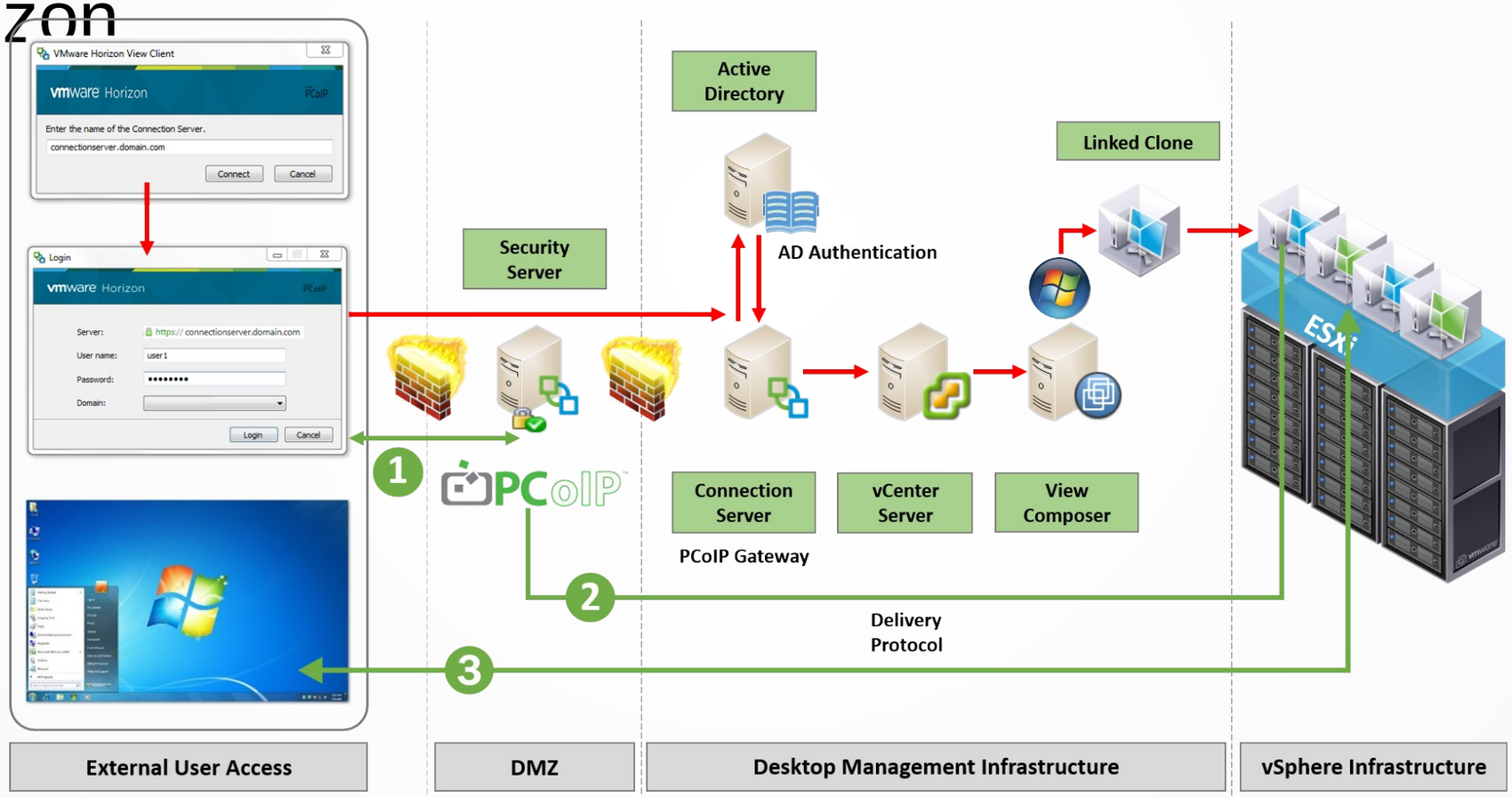
- Earn more (or spend less) money, of course !
  - To make multi-platform development easier.
  - To run old or exotic software on hardware you don’t have.
  - To optimize the usage of the hardware you possess.
  - To better isolate applications and improve security.
  - To better handle migrations and backups.
  - **To allows for user mobility.**
  - To play.
  - ...

To allows for user mobility.

- Your users may not have dedicated desktop computers. Instead they use whatever office and computer that is available.
- They still have their dedicated desktop environment running on a server (typically inside a container). The display is exported to a light terminal or a remote control application.
- Technology allows for sharing even GPU resources (thus allowing to run GPU intensive software in the cloud).

To allows for user mobility.

- In office or teaching environment, tools like VMWare Horizon



To allows for user mobility.

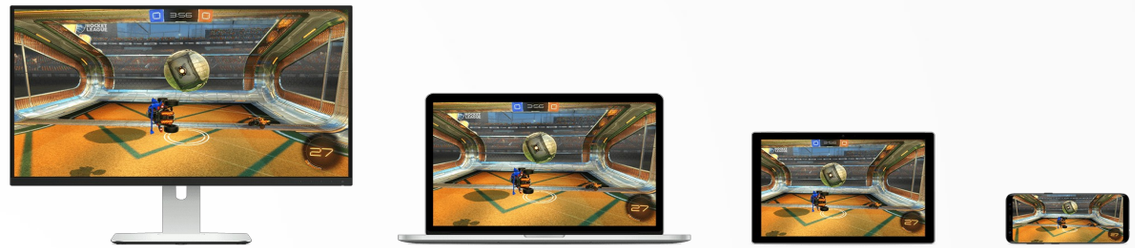
- Another strategy is to embed everything needed into a container then migrate / distribute this container wherever you want.
- The application find everything it needs inside the container. Ne more packages dependencies, DLL or third party software to install.
- The container is running on the user’s system, through virtualization
- Example: Windows containers

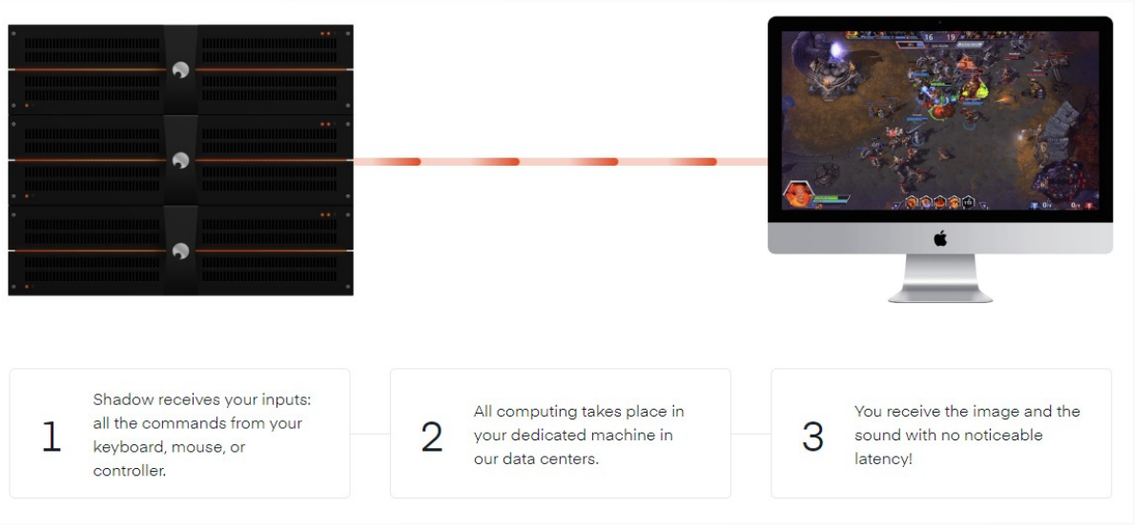
Benefits of virtualization

- Earn more (or spend less) money, of course !
  - To make multi-platform development easier.
  - To run old or exotic software on hardware you don’t have.
  - To optimize the usage of the hardware you possess.
  - To better isolate applications and improve security.
  - To better handle migrations and backups.
  - To allows for user mobility.
  - **To play.**
  - ...

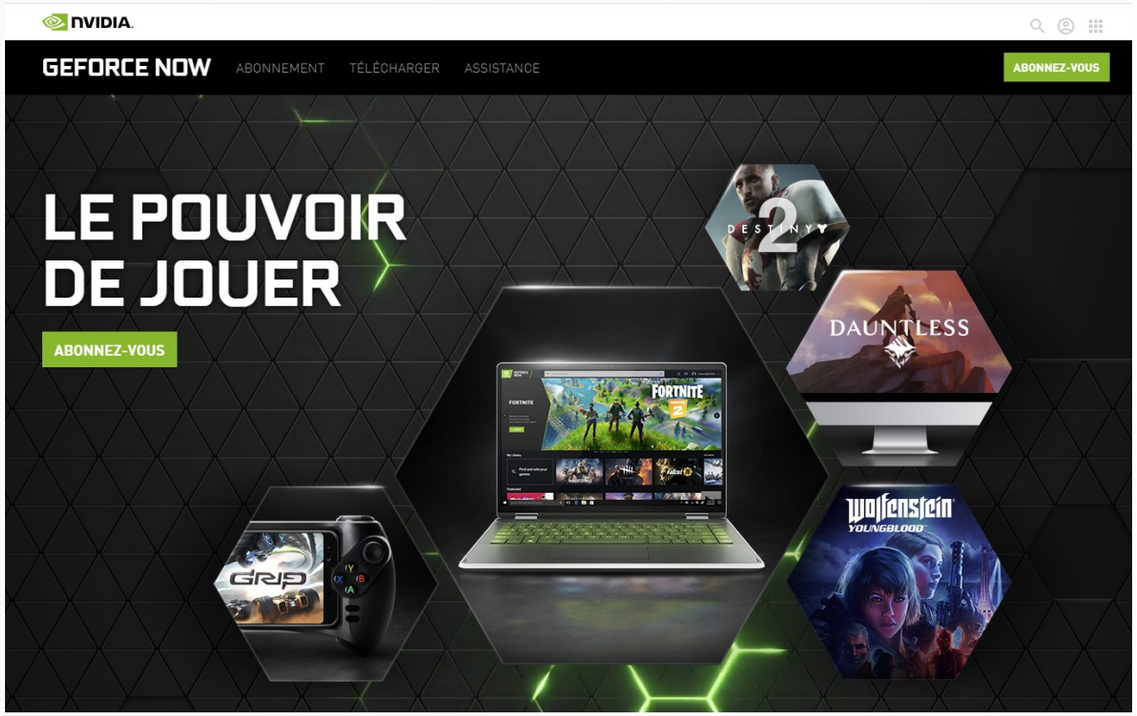
To play (and also for user mobility).

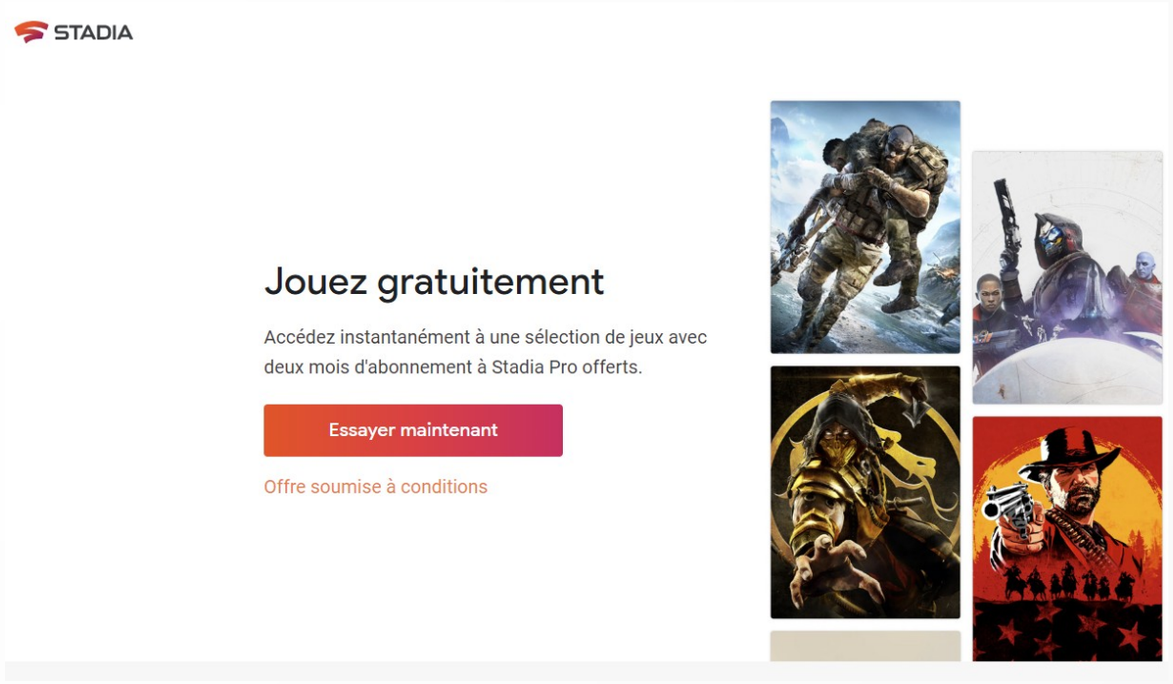
- New services that run games in the cloud and display them on whatever terminal you have
- Low latency, high bandwidth network required (typically optical fiber)
- Shadow, Google Stadia, GeForce Now, ...





To play (and also for user mobility).





Benefits of virtualization

- Virtualization allows for resources mutualization. Globally, you use:
  - Less power. As you make a better use of the hardware you have. Also, less power dissipated in computers means less power dissipated in cooling systems.
  - Less workforce. As management and maintenance are made easier. You can **streamline more procedures**, as you have less hardware diversity, less operating systems diversity, less drivers diversity, ...

Benefits of virtualization

- Security is simplified:
  - Because only one service is running per (virtual) host, most attacks and especially privileges escalation are less dangerous
  - Most services are managed using the same tools.
    - Users access lists / privileges
    - Firewall allowing only required communications between all entities
  - Putting firewall everywhere and analysing their logs give you a clear view of what is happening
  - If necessary you can cut off any compromised service independently.

Benefits of virtualization

- Strong backup policies:
  - By saving your VM in a working state, in case of failure you can restore them to a previously working state much faster. You don’t have to reinstall.
  - You can easily keep backup of multiple working state, and easily go back to any previous one.

Benefits of virtualization

- Ease of deployment:
  - You can use templating. You install and configure a VM to a desired working state, and then make copies every time you need a new server of this type.

    You then only have to change a few parameters (name, address, …). You create a new server in minutes, if not in seconds.
  - Cost (especially disk and even memory) can be mitigated using linked clones. Files common to multiple instances can be stored only once.

    When you have many identical servers used for load balancing, they have usually a lot in common.

Benefits of virtualization

- Scaling
  - Using templating, you can easily scale up your system as load increases. More users ? Then add more VMs !
  - Of course this means you have to set up load balancers than can distribute the load over your VMs.
- Disaster recovery
  - Yours physical servers don’t have to be in the same place.
  - By using multiple sites, you becomes much more resilient

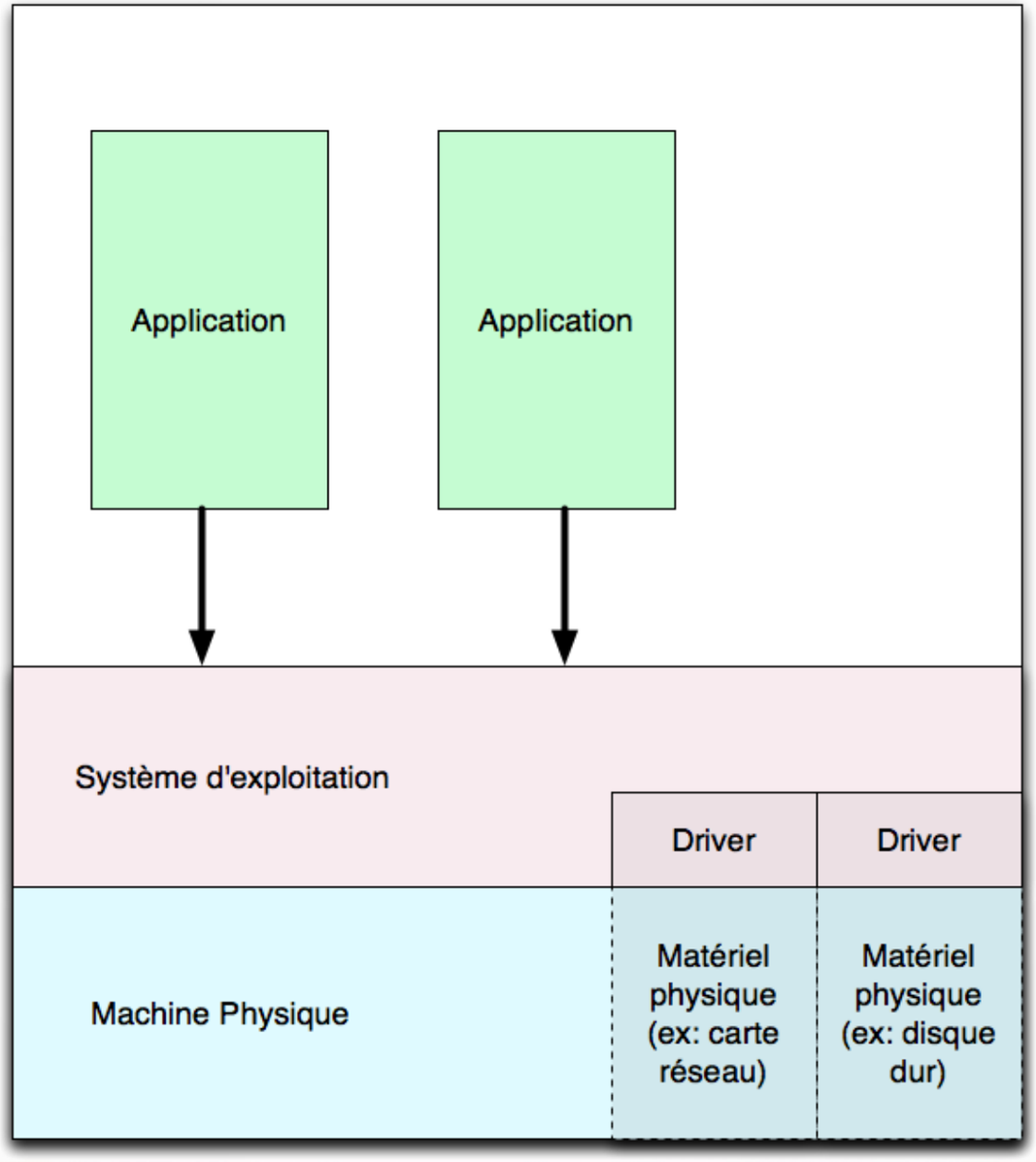
Benefits of virtualization

- Load balancing and global load balancing
  - You can use local load balancers to handle
    - Local load increase
    - Maintenance (when shutting down a server for maintenance, load is redirected to other servers).
  - You can use global load balancing to balance between sites
    - Necessary in case of catastrophic failure on a site
  - Global load balancing is complex
    - You can use it to direct users to geographically close sites (for performances increase)
    - But you often have to mess with DNS (have your own)
    - You have to be careful to have required user data available at the site you send him.
    - You have to be careful of data integrity between sites

Types of virtualization

- Multiple forms of virtualization exists, each having their advantages and drawbacks.
- Knowing their forces and weaknesses is mandatory to choose the right type for you application or architecture.

Without virtualization



Without virtualization

- The operating system manage the hardware
  - For specific hardware, it requires drivers provided by hardware manufacturers
- The operating system provide APIs (Application programming Interfaces) to applications
  - For filesystem access
  - For display access
  - For network access
  - ...

Without virtualization

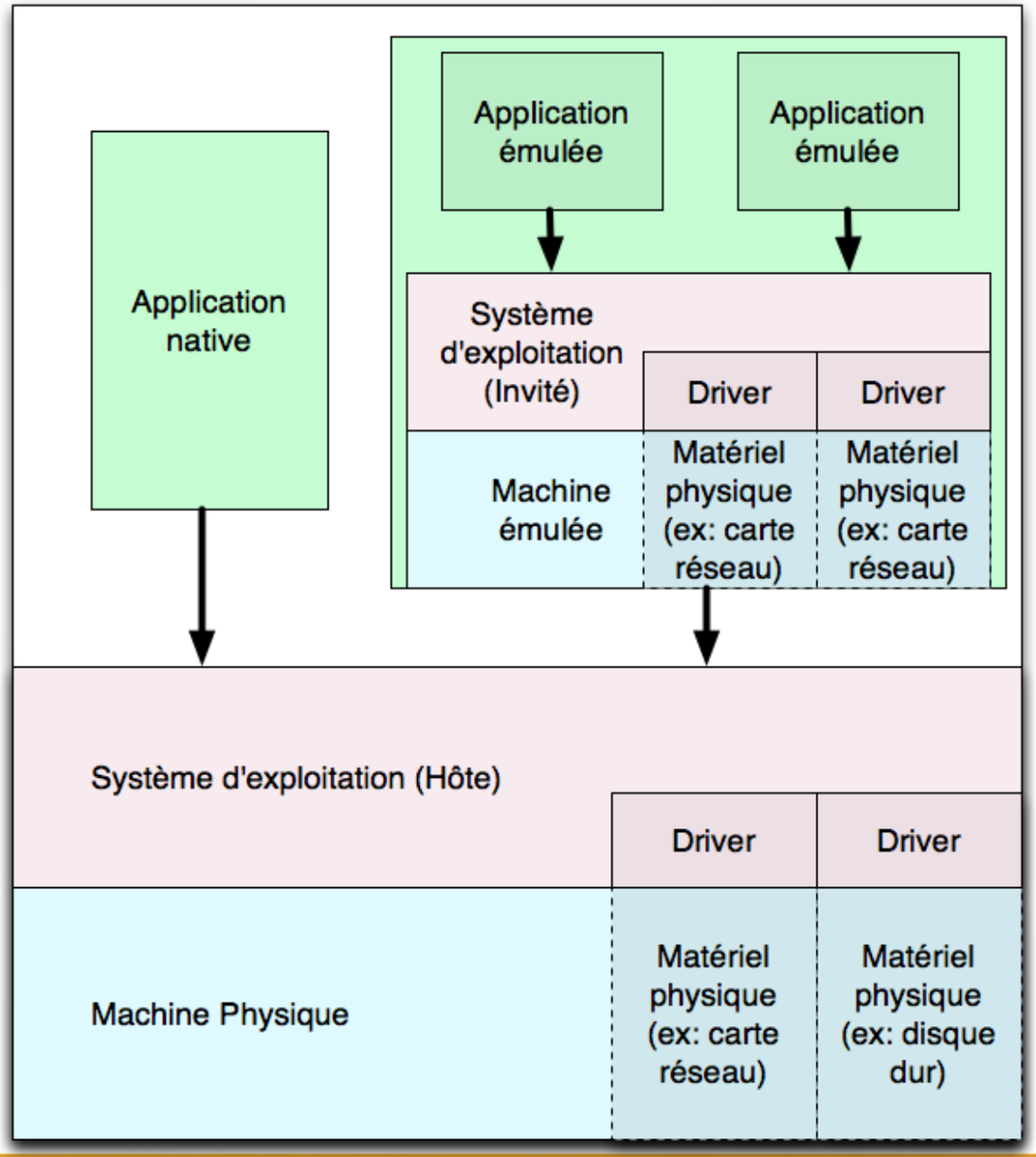
- Applications are compiled
  - For this specific hardware architecture. It means the executable file contains opcodes recognized by the processor of this computer.
  - For the API of this operating system. When the application want to read a file, it has to ask the operating system in the correct way (i.e. using the correct API).

    For example, reading a file on windows is done using a certain set of function. On Linux, it is done using a different set of functions.
  - Applications build for the physical architecture and operating system of a computer are called **native** applications.

Without virtualization

- Native application works only on this physical architecture **and** with this operating system.
- Native applications have the best performances. They are the standard against which performances of the various types of virtualization will be compared.

Complete emulation



Complete emulation

- The operating system is the same as without virtualization. It manages the hardware and provide APIs to applications.
- It is still able to run native applications.
- An emulator is a native application running on this system.
- The operating system of the physical computer will be called the **host** operating system.

Complete emulation

- The emulator simulates a complete hardware system
- To be usable, this simulate hardware has to be managed by an operating system. So this **guest** operating system has to be installed inside the emulator. It even need drivers to handle the simulated hardware.
- The guest operating system also provide APIs to guest applications.

Complete emulation

- The simulated hardware may or may not be of the same type as the physical host.
  - If you emulate a game console or a smartphone on a PC, you are emulating a very different architecture.
    - Each instruction in the guest application code has to be analysed and translated into one or more instructions executable by the host system
    - Sometime, hardware capabilities of the guest system have to be implemented in software.
    - You also have to execute all the code of the guest operating system.
    - All of this incurs a great cost in performances. That may or may not be a problem. Emulating a Gameboy on a modern computer is not a problem. Emulating the upcoming playstation 5 will be … quite difficult, for a while.

Complete emulation

- The simulated hardware may or may not be of the same type as the physical host.
  - You can also emulate an architecture similar to the physical one.
    - Most instructions in the guest will not have to be translated. As we will see later, only some of them (especially memory accesses and I/O) will require a special handling.
    - Even if you emulate the same architecture, you can need a guest operating system. It may or may not be similar to the host operating system.
      - For exemple, on a PC (physical architecture) running windows, you can create a VM emulating a PC running linux.
    - As the architecture is the same, this simplify the translation process and the emulator run faster. Depending on the guest application, it can run close to the speed of a native application.

Complete emulation

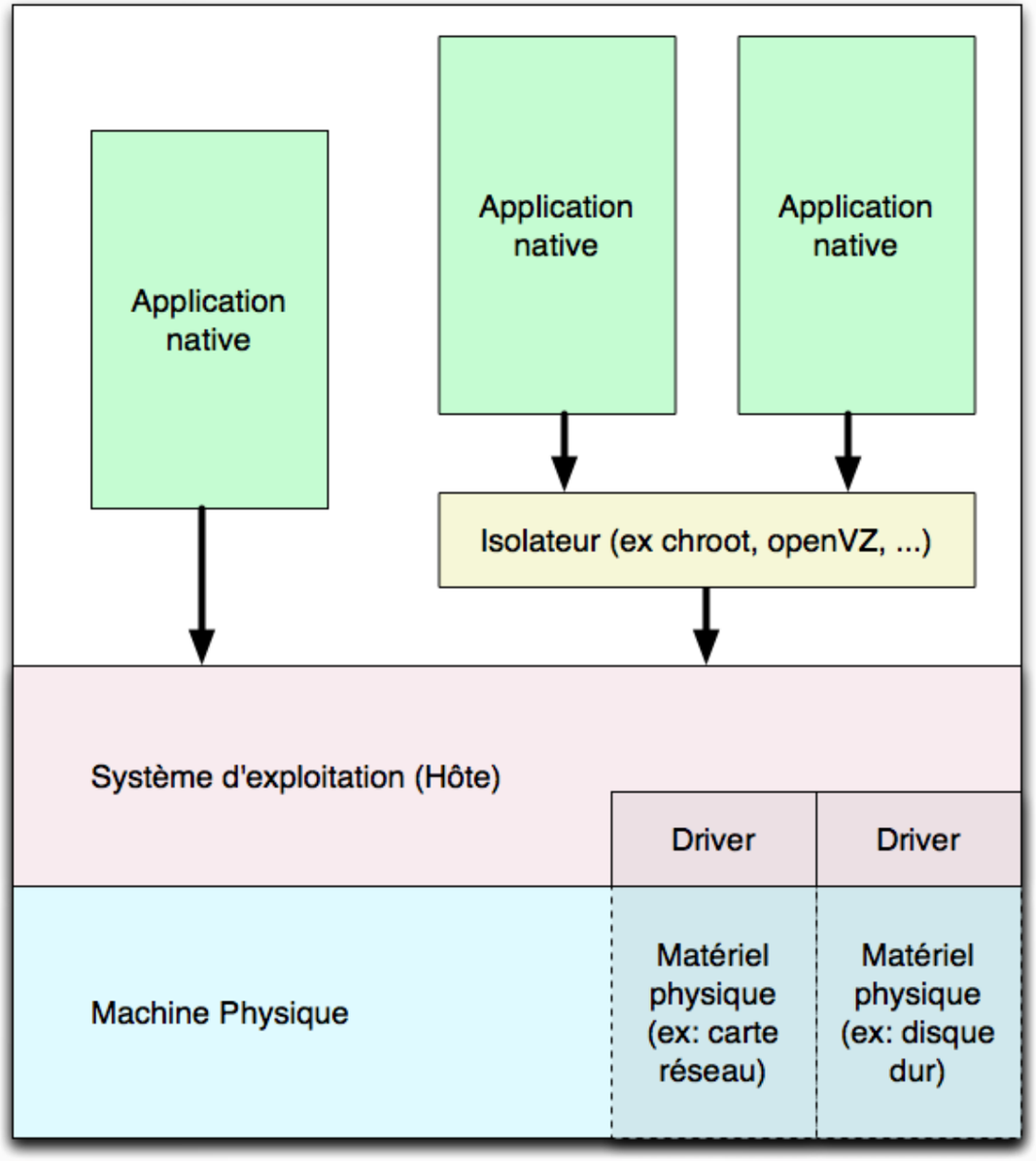
- As you have to install a guest operating system, there is always a great costs tied to full emulation.
  - Memory.
  - Storage (disk) space.
- Complete emulation is the most versatile technique
  - You can emulate any architecture and guest Operating System on any other host and operating system (provided the host is fast enough).

Complete emulation

- Performances depends on
  - The difference in host and guest physical architecture. Same architecture means much faster emulation.
  - If the architectures are identical, then the nature of the guest application also has in impact. Computation instructions can be executed without modification (i.e. at native speed), whereas memory accesses and some other instructions need special handling.

    Application that mostly do computations can run almost at native speed.

Isolators and containers



Isolators and containers

- We have seen that a key benefit of virtualization is a better **isolation**. For security reasons, we want applications to be as isolated as possible, ideally each on its own computer.
- We can build **isolators** to do just that.
- When using an isolator, guest applications are just native applications, compiled for the host architecture and operating system.
- We interpose an isolation component between the operating system and the guest application.

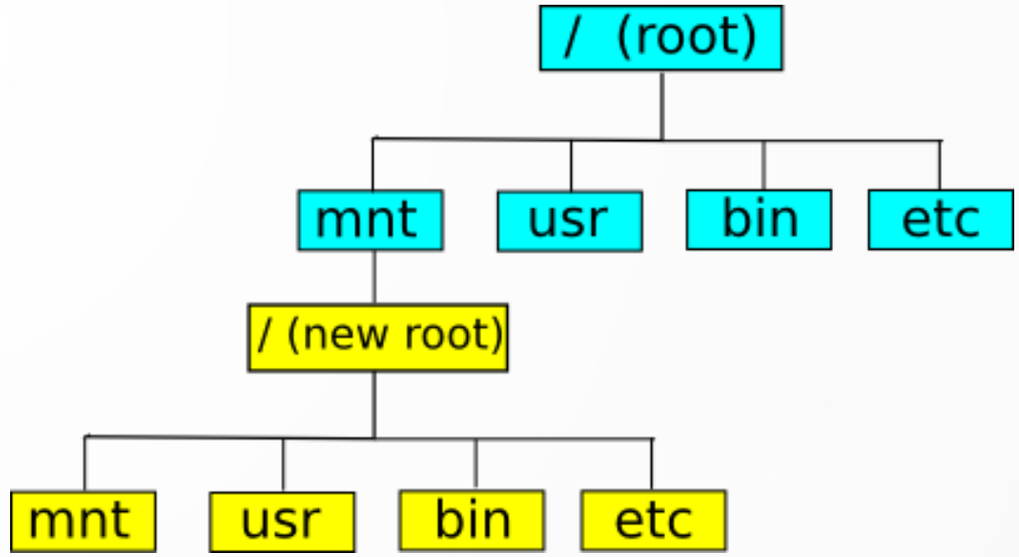
Isolators and containers

- Normally, applications communicate directly with the operating system trough its APIs.
- But when using an isolator, APIs calls have to go trough the isolator, which can filter and modify on the fly what the application can ask to or receive from the operating system.

Isolators and containers

- A basic isolator example is “chroot”, that alter the parts of the filesystem effectively visible and accessible to a guest application.

- The “jailed” application can only access the yellow part of the filesystem. For example, when trying to open /toto.txt, the isolator redirect the path to “/mnt/toto.txt” without letting the application know.



Isolators and containers

- More advanced isolators are called containers. They mask much more of the host's systems
  - Filesystem
  - Other processes (running the "ps aux" command inside a container will not list the applications running ouside)
  - Memory. Quotas can be enforced at container level. Application only see the amount of memory available to them.
  - Network. The container may create virtual interfaces carrying their own addresses. Application inside this container see only those interfaces and can be accessed trough them. A the host level, thos virtual interfaces are bridged to the physical one(s).

Isolators and containers

- Containers and isolators have advantages:
  - They cause way less computation overhead compared to full virtualization, as only one operating system (the one from the host) is running.
  - Performances are usually very close (a few percents less) to pure native applications. In fact, guest applications are native applications and only some system calls have to be intercepted and modified on the fly.
  - The required much less memory (no guest operating system to load) and much less disk space.

Isolators and containers

- Containers and isolators have advantages:
  - As there is only one operating system (especially, one kernel), when you upgrade or apply security patches, it affects all containers. It becomes easier to maintain all containers up-to-date
  - If you want specific libraries for a given application, you can install them inside its container. This way, different applications can have different libraries that do not interfere.

Isolators and containers

- Containers and isolators have advantages:
  - They are easy and fast to create. Usually we use an archive (zip file) containing a file tree with all the necessary libraries, tools and configuration files. When creating a new container, we mostly have to decompress this archive and change a few configuration files (hostname, ip address, …). It’s much faster than installing a new operating system.
  - It’s also easy to convert a running container into a template (the archive file), allowing the easily clone a working setup.

Isolators and containers

- Containers and isolators have advantages:
  - They boot really fast ! In fact starting a container mostly means you start a few more processes on an already running operating system. It’s much faster than completely booting a new one !

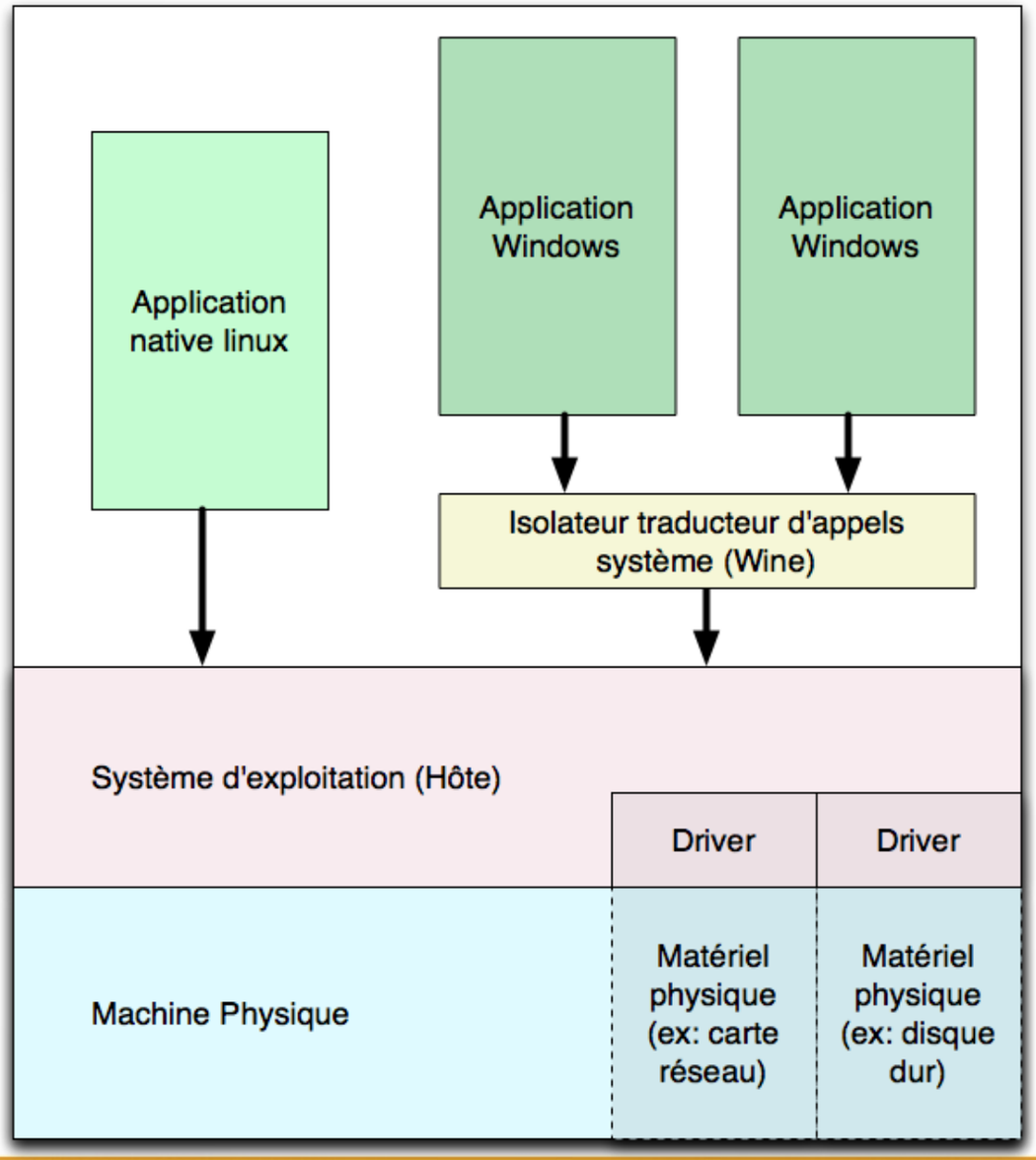
Containers and isolators have drawbacks and limitations:

- Containers and isolators have drawbacks:
  - The guest applications have to be native. They have to be compiled for this architecture and host operating system. You can’t run windows applications on Linux this way. If you are renting a low cost Virtual Private Server, it most probably will be providing containers, and you will be limited to its operating system.
  - In advanced containers like LXC (LinuX Containers), libraries are inside the container. This can become an headache to maintain. You have to go through all containers and make sure there is no problem for them to work with the current host kernel.

Containers and isolators have drawbacks and limitations:

- Containers and isolators have drawbacks:
  - As applications are in reality native ones, live migration is usually not possible. You can move a container from one server to one another, but you have to stop it first. Note that it can restart really fast, as you don’t boot a full operating system.
  - Isolation can be considered one step lower than full virtualization. If by some way you gain access to the kernel, then you have full access to all containers running on the same computer.

Translating system calls



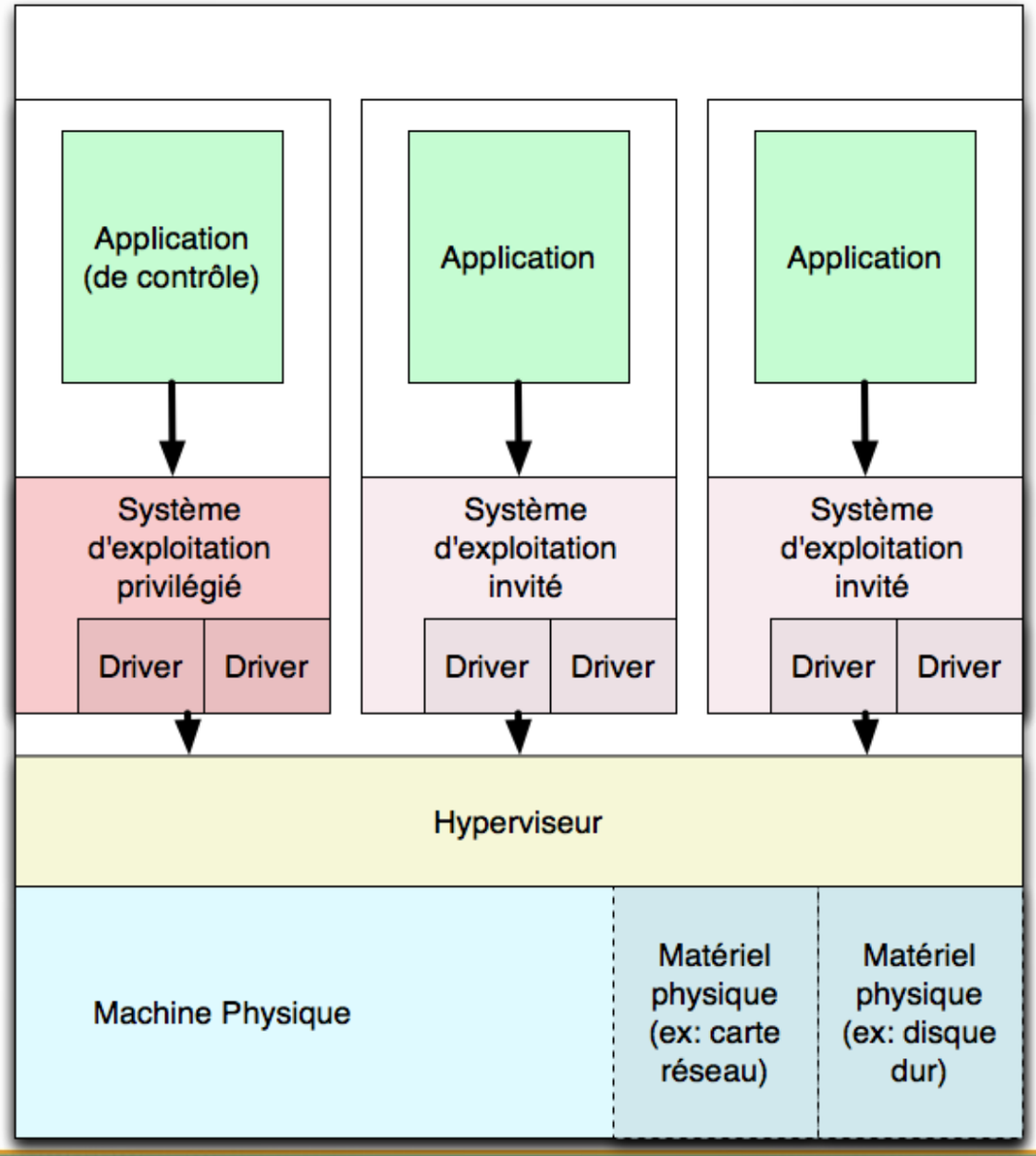
Translating system calls

- Containers and isolators offer very good performances, but limit you to native applications.
- The isolator can be transformed so that it translates system calls. In fact, Windows and Linux system call are quite similar in most functionalities, but different in syntax.
- Translating system call allows to run applications that were compiled for this physical architecture but for a different operating system.

Translating system calls

- Performances depend on the nature of the application.
  - When doing mostly computation, speed is similar to native one.
  - Complex system calls (think translating Direct3D into OpenGL for example), costs more.
- **Wine** allows to run Windows binaries on Linux
- **Windows Subsystem for Linux (WSL)** allows to run Linux binaries on Windows. Note that the 2019 version (WSL2) now use full virtualization and thus run a true Linux kernel to enhance compatibility.

Hypervisor


Application
(de contrôle)
Application
Application
Système
d'exploitation
privilégié
Système
d'exploitation
invité
Système
d'exploitation
invité
Driver
Driver
Driver
Driver
Driver
Driver
Hyperviseur
Machine Physique
Matériel
physique
(ex: carte
réseau)
Matériel
physique
(ex: disque
dur)

Hypervisors

- Hypervisors are very basic systems that mostly manage the hardware and allow Virtual Machines to coexist.
- We call them “bare metal” as they directly interact / manage the hardware.
- They usually do not provide the full range of services we find in operating systems. Consequently, to use the computer, we have to install a privileged operating system, that can provide user interface and other high level services.

Paravirtualization

- In full virtualization, guests are not normally aware they are inside a virtual machine.
- They use their normal drivers to manage the simulated hardware.
- This is best for isolation, but may lead to poor performances. In particular, they are not able to benefits to all the capabilities of underlying hardware.

Paravirtualization

- It is possible the install special drivers inside the guest, that “break” the isolation by being aware of the virtual machine.
- These drivers interoperate with the virtualization software, that let them access the real hardware.
- The performance increase can be drastic.
- Those are called **paravirtualized** drivers.

Paravirtualization

- Paravirtualized drivers are not always available.
  - The have to be written to work with a specific virtualization software (for example: paravirtualized drivers for VirtualBox)
  - The have to be written for the Operating System you are using inside your virtual machine (for example: windows drivers).
- They can be provided:
  - By the virtualization software vendor
  - Included in the Operating System tou want to install (Linux include a set of paravirtualized drivers, that help to install Linux inside virtual machines).

Paravirtualization

- Paravirtualized drivers may provided additional services.
  - Desktop integration with between host and guest (allowing copy & paste, …)
  - Allow to mount virtual drives in the VM that point to directories of the host.
  - In some cases, allow guest applications to run in their own window in the host system.